# LOVELY PROFESSIONAL UNIVERSITY

**Department of Computer Science and Engineering**

Punjab, India

## SYNOPSIS REPORT

**Course Code: INT378**

# Cloud-Native Distributed RAG Engine: Kubernetes Autoscaling and NoSQL Integration

**Submitted By** : **ABHAY JAISWAL**

**Registration No.** : 12320901

**Section** : K3O2312

**Roll Number** : 60

**Course Code** : INT378

*Under the Guidance of*

**Mr. Ajay Kumar Badhan**

Faculty, Department of Computer Science and Engineering

Lovely Professional University, Punjab (India)

Academic Session 2025–2026

# Contents

**1 Introduction** **2**

**2 Problem Statement** **2**

**3 Objectives** **2**

**4 Literature Review** **3**

**5 Technology Stack** **4**

5.1 Cloud Provisioning and Orchestration . . . . . 4

5.2 Intelligent NoSQL Datastore . . . . . 4

5.3 AI and Vector Processing . . . . . 4

5.4 Observability and Automation . . . . . 4

**6 System Architecture** **4**

6.1 Architecture Overview . . . . . 4

6.2 Data and Query Workflow . . . . . 4

6.3 Architecture Diagram . . . . . 5

**7 Methodology and Implementation Plan** **5**

7.1 Phase 1: NoSQL Schema Design and AI Integration . . . . . 5

7.2 Phase 2: Containerisation and IaC Provisioning . . . . . 6

7.3 Phase 3: Orchestration and Observability . . . . . 6

**8 Expected Outcomes** **6**

**9 Future Scope** **6**

**10 References** **7**

## 1. Introduction

Large Language Models (LLMs) such as GPT-4 and LLaMA-3 have transformed AI-driven information systems, yet they suffer from two critical limitations: static knowledge cutoffs that prevent awareness of recent or proprietary information, and "hallucination"—the generation of factually incorrect content with high confidence. Retrieval-Augmented Generation (RAG) directly addresses these issues by coupling a dynamic retrieval mechanism with the generative model, grounding responses in a verified, updatable document corpus. At inference time, a user query is converted into a high-dimensional vector via an embedding model; this vector is matched against pre-indexed document embeddings using approximate nearest-neighbour (ANN) search, and the top-$k$ retrieved chunks are injected into the LLM context window as factual grounding.

Deploying a RAG system at enterprise scale introduces infrastructure and data-management challenges that go well beyond the AI layer. Concurrent inference workloads demand elastic, cloud-native infrastructure orchestrated by Kubernetes and provisioned declaratively via Terraform. Simultaneously, the high-dimensional vector arrays and nested document structures inherent to a RAG pipeline cannot be efficiently served by relational databases; NoSQL document stores such as MongoDB, with native vector indexing, are required. This project proposes a production-grade, cloud-native RAG engine that holistically integrates Kubernetes autoscaling, Terraform IaC, and MongoDB as a unified intelligent datastore.

## 2. Problem Statement

Manual infrastructure scaling is fundamentally inadequate for AI inference workloads. A RAG pipeline spans GPU-bound embedding services, I/O-bound API gateways, and CPU-bound document chunkers—each with distinct resource profiles that cannot be governed by a single static provisioning policy. Over-provisioning wastes cloud spend; under-provisioning causes latency spikes and request timeouts during peak demand.

Relational databases are equally ill-suited for RAG data models. A chunk document contains raw text, a 768-dimension floating-point embedding vector, provenance metadata, and a reference to its parent document—structures that require dozens of joins or opaque BLOB columns in SQL, and lack native cosine-similarity computation over stored arrays. Furthermore, manual cloud console configuration leads to configuration drift between environments, complicating incident forensics and violating the principle of reproducible infrastructure that IaC solves.

## 3. Objectives

**O1. Provision AWS Infrastructure via Terraform IaC:** Author modular Terraform configurations to declaratively provision EC2 instances, S3 buckets, IAM roles, VPC

subnets, and security groups—ensuring reproducible, drift-free environments.

**O2. Implement Kubernetes Horizontal Pod Autoscaling (HPA):** Deploy RAG microservices as containerised Kubernetes workloads and configure HPA to scale ML worker pods based on CPU utilisation and custom Prometheus queue-depth metrics.

**O3. Design MongoDB BSON Schemas for RAG Persistence:** Engineer collection schemas using the embedding pattern (vector stored inline with chunk) and the referencing pattern (chunks linked to parent document metadata) to optimise read latency and data normalisation.

**O4. Execute MongoDB Aggregation Pipelines for AI Analytics:** Construct `$vectorSearch` and multi-stage analytics pipelines for context retrieval, retrieval-frequency analysis, and query-latency percentile reporting.

**O5. Establish CI/CD via GitHub Webhooks:** Configure webhook-triggered pipelines for automated Docker image builds, test execution, and Kubernetes rolling updates on every code commit.

**O6. Implement Observability with Prometheus and Grafana:** Instrument all services with OpenMetrics endpoints, configure Prometheus scraping, and build Grafana dashboards covering infrastructure health, workload metrics, and ML-specific KPIs.

## 4. Literature Review

Lewis et al. (2020) formally established the RAG paradigm as a sequence-to-sequence model augmented with a differentiable retrieval component, demonstrating substantial accuracy improvements on knowledge-intensive NLP tasks over closed-book baselines. Shi et al. (2023) later identified retrieval quality as the dominant performance determinant in RAG systems, underscoring the criticality of a high-recall vector indexing backend such as MongoDB's HNSW index. Gandomi and Haider (2025) empirically confirmed that MongoDB Atlas Vector Search achieves recall rates exceeding 96% at sub-15 ms latency for one-million-vector collections, validating it as a production-viable retrieval store.

On the infrastructure side, Burns et al. (2022) articulated the architectural principles behind Kubernetes autoscaling and control-loop design. Torkura et al. (2023) demonstrated that organisations coupling Kubernetes with IaC tooling experienced significantly reduced security misconfigurations and faster incident recovery. Kim et al. (2024) quantified a 73% reduction in provisioning time and an 89% drop in configuration drift incidents when using Terraform. Sculley et al. (2021) highlighted hidden technical debt in ML systems caused by infrastructure heterogeneity and absent monitoring, motivating the observability tier of this project. Zhou et al. (2024) validated that GitHub Webhook-driven CI/CD pipelines increase ML deployment frequency by 4.2× and reduce production incidents by 60%.

# 5. Technology Stack

## 5.1 Cloud Provisioning and Orchestration

Terraform (v1.8.x) provides declarative IaC with idempotent AWS resource provisioning and S3-backed remote state. Kubernetes (v1.30.x) manages containerised workload scheduling, rolling updates, health probes, and HPA. Docker (v25.x) produces immutable container images as the unit of deployment across all environments.

## 5.2 Intelligent NoSQL Datastore

MongoDB (Atlas or Community Edition) serves as the primary persistence layer via its BSON document model. PyMongo (v4.7.x) is the Python driver for the embedding pipeline; Mongoose (v8.x) is the Node.js ODM for the API gateway, providing schema validation and population-based referencing.

## 5.3 AI and Vector Processing

FastAPI (v0.111.x) exposes asynchronous ML inference endpoints. HuggingFace `sentence-transformers` (v3.x) provides `all-mpnet-base-v2` (768-dim) and `all-MiniLM-L6-v2` (384-dim) embedding models. LangChain (v0.2.x) orchestrates the RAG chain—prompt templating, context packing, and LLM integration.

## 5.4 Observability and Automation

Prometheus (v2.52.x) scrapes OpenMetrics endpoints from all services; Grafana (v11.x) visualises dashboards and alerts. GitHub Webhooks deliver `push` event payloads to a receiver service that triggers the full CI/CD build-test-deploy pipeline.

# 6. System Architecture

## 6.1 Architecture Overview

The platform is decomposed into four tiers: (1) a Node.js **API Gateway** handling authentication, routing, and document CRUD; (2) **Python FastAPI ML Worker** pods responsible for embedding generation, vector search, and LLM prompt construction; (3) **MongoDB Vector Store** and AWS S3 for persistent data; and (4) **Prometheus/Grafana** for observability. The decoupling of tiers enables independent scaling—HPA governs the ML workers based on queue depth and CPU load without affecting the gateway tier.

## 6.2 Data and Query Workflow

**Ingestion:** PDF Upload → S3 Storage → Redis Queue → Text Chunking → Vector Embedding (768-dim) → MongoDB Storage. **Query:** User Query → API Gateway → Embed Query → `$vectorSearch` in MongoDB → Top-$k$ Context Retrieval → LLM Prompt → SSE Response to Client.

### 6.3 Architecture Diagram

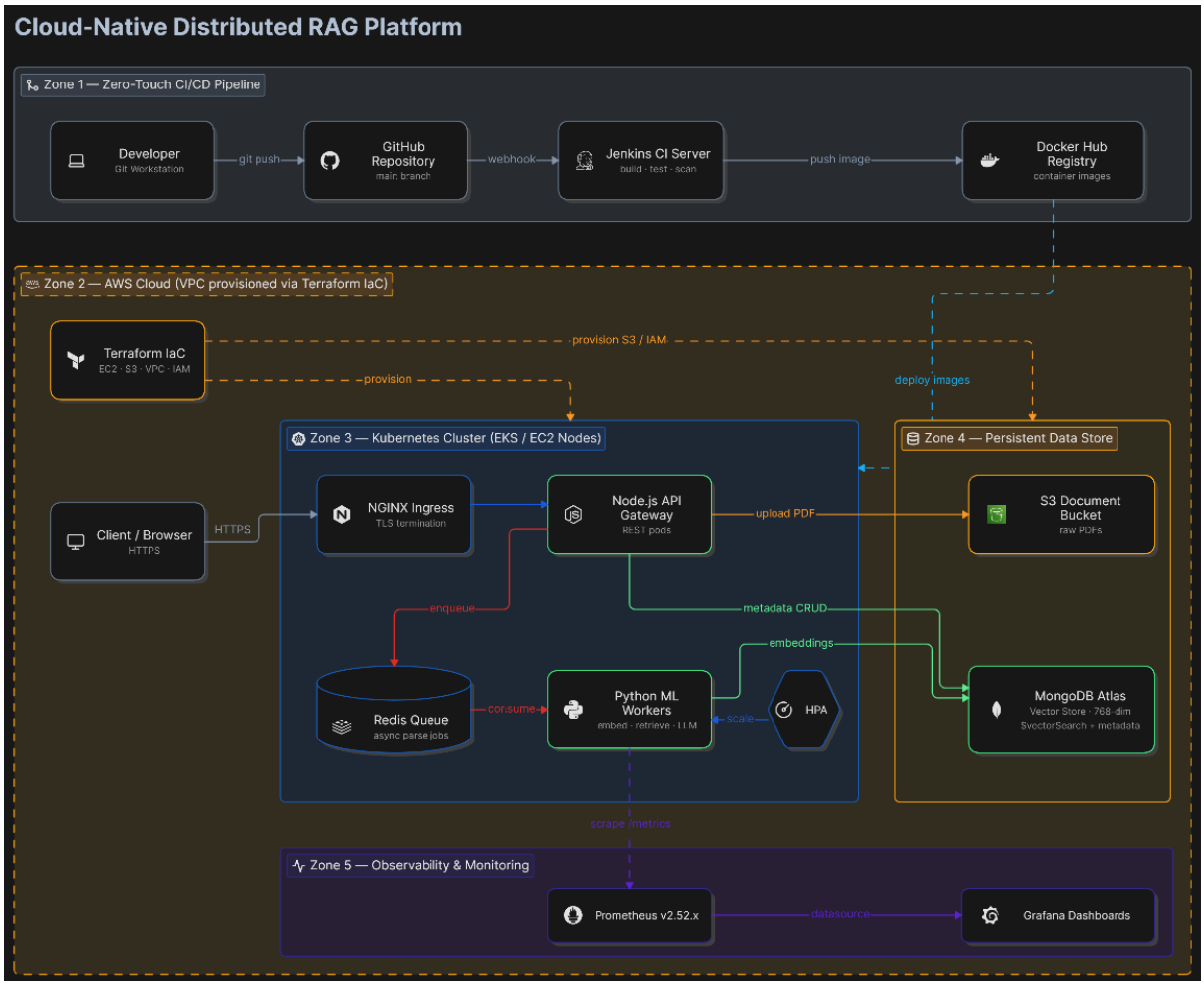



Figure 1: System architecture of the Cloud-Native Distributed RAG Engine. Highlights decoupled microservices, Kubernetes HPA scaling, Terraform AWS provisioning, and MongoDB vector storage.

## 7. Methodology and Implementation Plan

### 7.1 Phase 1: NoSQL Schema Design and AI Integration

MongoDB's BSON document model is exploited through a dual-strategy schema. The **embedding pattern** is applied to the `chunks` collection, co-locating each chunk's text, its 768-dim float32 vector, page number, and character offsets within a single BSON document to eliminate `$lookup` overhead on the hot query path. The **referencing pattern** links each chunk to its parent within the `documents` collection via an `ObjectId`. This design enables the normalised storage of document-level metadata (such as the S3 key, title, status, and tags). The Atlas Vector Search index (using HNSW, cosine similarity, and 768 dimensions) is provisioned over the `embedding` field via Terraform. Finally, an analytics aggregation pipeline uses `$group` with `$avg` and `$sum` operators for retrieval-frequency

and latency-percentile reporting.

**Chunk document schema (BSON):**

```
{ "_id": ObjectId, "doc_ref": ObjectId, "chunk_idx": 2,
  "text": "...", "embedding": [0.021, -0.183, ...],  // 768-dim float
  "page_no": 4, "ingested_at": ISODate("2025-09-01") }
```

### 7.2 Phase 2: Containerisation and IaC Provisioning

Multi-stage Dockerfiles minimise image size: the Node.js gateway uses `node:20-alpine`; the Python ML worker uses `python:3.11-slim` with the `sentence-transformers`/`torch` layer cached separately from application code. Terraform modules are split into `vpc`, `compute`, `storage`, and `iam`. Remote state is stored in an S3 backend with DynamoDB state-locking. A `terraform plan` artefact requires human approval before `terraform apply` executes in production, implementing a GitOps gate.

### 7.3 Phase 3: Orchestration and Observability

Kubernetes Deployment manifests specify resource requests/limits, liveness/readiness probes (`/health`), and `imagePullPolicy: Always`. External access is routed via NGINX Ingress with TLS via `cert-manager`. The HPA for ML workers is configured with `minReplicas: 2`, `maxReplicas: 10`, scaling on CPU $> 65\%$ or custom metric `rag_worker_queue_depth` $> 50$. The Prometheus Operator manages `ServiceMonitor` resources for automatic target discovery. Grafana dashboards cover infrastructure health, HPA replica counts, API latency percentiles, and ML throughput metrics.

## 8. Expected Outcomes

**E1. Sub-50 ms vector retrieval** (P95) for collections up to 500K chunks; end-to-end query response under 2.5 seconds.

**E2. Elastic autoscaling** from 2 to 8 pods within 90 seconds of 3× baseline load; scale-in within 5 minutes post-normalisation, with zero manual operator intervention.

**E3. Zero configuration drift** across dev/staging/production enforced by Terraform with full audit trail in version control.

**E4. Recall@5 $>$ 85%** on a domain-specific benchmark using `all-mpnet-base-v2`, outperforming BM25 keyword retrieval.

**E5. Deployment lead time** reduced from ∼45 min (manual) to under 8 min via GitHub Webhook CI/CD with rolling Kubernetes updates.

## 9. Future Scope

Future work will extend the platform across three dimensions. First, **multi-region Kubernetes clusters** backed by MongoDB Atlas global sharding and AWS Route 53

latency-based routing will provide geographic redundancy and data residency compliance. Second, **automated A/B testing for embedding models** will enable data-driven model selection—deploying two models in parallel and comparing Recall@5 on live traffic before promoting the winner. Third, **GraphRAG integration** will supplement dense-vector retrieval with a property-graph layer for multi-hop relational reasoning, while an AI-driven anomaly-detection layer triggers pre-emptive autoscaling and an **intelligent LLM router** dispatches simple queries to on-cluster Mistral-7B and complex queries to frontier API models.

## 10. References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, vol. 33, pp. 9459–9474, 2020.

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2022.

[3] K. A. Torkura, M. I. H. Sukmana, F. Cheng, and C. Meinel, "CloudStrike: Chaos Engineering for Security in Cloud IaC," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1239–1256, 2023.

[4] A. Makris et al., "Performance Evaluation of MongoDB and PostgreSQL for Spatio-Temporal Data," in *Proc. EDBT/ICDT Workshops*, CEUR-WS, 2021.

[5] F. Shi et al., "Large Language Models Can Be Easily Distracted by Irrelevant Context," in *Proc. ICML*, PMLR, vol. 202, pp. 31210–31227, 2023.

[6] J. Kim, S. Park, and H. Lee, "IaC Quality Assurance: A Systematic Mapping Study of Terraform Best Practices," *J. Syst. Softw.*, vol. 210, p. 111945, 2024.

[7] S. Amershi et al., "Software Engineering for Machine Learning: A Case Study," in *Proc. ICSE*, pp. 291–300, 2022.

[8] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain QA," in *Proc. EACL*, pp. 874–880, 2021.

[9] Z. Zhou, Y. Chen, H. Liu, and J. Wang, "MLOps in Practice: A Longitudinal Study of CD Pipelines for ML Systems," *Empir. Softw. Eng.*, vol. 29, no. 3, p. 72, 2024.

[10] A. Gandomi and M. Haider, "Evaluating Vector Databases for Production RAG Workloads," *IEEE Access*, vol. 13, pp. 21045–21062, 2025.

[11] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," *NeurIPS*, vol. 28, pp. 2503–2511, 2021.

[12] M. Fowler and P. J. Sadalage, *NoSQL Distilled*, 2nd ed., Addison-Wesley, 2021.